National University of Computer and Emerging Sciences

Karachi – Campus

Introduction to Information and Communication Technologies (CL1000)

Lab Final-Exam

Total Time (Hrs): 02
Total Marks: 50
Total Questions: 04

Paper A

Date: December 05, 2024

Time: 08:30 AM – 10:30 AM

Course Instructor(s)

Mr. Ghulam Qadir Bhurgari, Miss. Khadija tul Kubra

Student Name	Section	Roll Number

Q1: (Marks: 10)

You are tasked with creating an **Inventory Management System** that allows users to manage products in a store, update product quantities, and calculate the total value of products in stock. The system tracks the products, their quantities, and their prices, and allows users to interact with them using simple functions.

Requirements:

1. **Data Structure**:
   - Use an array to store the list of products.
   - Each product has the following attributes:
     - productName (string): The name of the product.
     - quantity (integer): The quantity of the product in stock.
     - price (float): The price of the product.
2. **Functions**:
   - **addProduct(productName, price)**: Adds a new product to the inventory with a starting quantity of 0.
   - **updateQuantity(index, quantity)**: Updates the quantity of the product at the specified index.
   - **sellProduct(index, quantity)**: Decreases the quantity of a product by the specified quantity when the product is sold.
   - **displayInventory()**: Displays all products with their names, quantities, and prices.
   - **calculateTotalValue()**: Calculates and displays the total value of all products in stock (quantity * price).
3. **Program Flow**:
   - Users can add products using the addProduct function.
   - Users can update the quantity of a product using the updateQuantity function.
   - Users can sell products, which reduces the quantity in stock, using the sellProduct function.
   - The system allows users to view the current inventory using the displayInventory function.
   - The total value of the inventory is calculated and displayed using the calculateTotalValue function.

Output

```
Product added: Laptop, Price: 1000.00
Product added: Mouse, Price: 25.50
Product added: Keyboard, Price: 45.75

Updated quantity of Laptop: 10
Updated quantity of Mouse: 50
Updated quantity of Keyboard: 30

Sold 5 units of Laptop. Remaining stock: 5
Sold 10 units of Mouse. Remaining stock: 40

Displaying Inventory:
1. Laptop, Quantity: 5, Price: 1000.00
2. Mouse, Quantity: 40, Price: 25.50
3. Keyboard, Quantity: 30, Price: 45.75

Total value of inventory: 17610.00
```

Q2: **[wgt: 15]**

You are required to create a **tourism-themed webpage** using HTML and inline CSS. First, create a file called tourism_exploration.html with a complete HTML structure and title the page as **"Discover New Destinations!"** The page should contain several sections, each with specific styles applied directly through inline CSS. The webpage should include the following:

1. **Hero Section**: The H1 heading should read **"Discover New Destinations!"** with the following style:
   - Light blue text
   - Font size of 48px
   - Center-aligned text
   - 80px top margin
   - A background image should be used to enhance the look, covering the entire section.
2. **Navigation Bar:** The navigation bar should contain several anchor links to different sections on the page (e.g., Why Travel? Top Destinations, Travel Tips, Connect, Login). Ensure the following:
   - Each link should have a 15px margin.
   - Use a dark blue background for the navbar.
   - The **Login button** should be highlighted in bold with a background color of orange (#ff8000) and padding of 10px 20px. It should be positioned on the right side of the navbar.
   - Use inline CSS to style the links with appropriate font sizes (18px) and colors (white).
3. **Why Travel Section:**
   - The H2 heading should read **"Why Travel?"** with dark blue text, a font size of 36px, center-aligned text, and a 20px bottom margin.
   - The section should have a background color of light gray (#f9f9f9) with 50px padding around the content.
4. **Top Destinations Section:**
   - The H3 heading should read **"Top Destinations to Explore"** with dark green text, a font size of 32px, and italicized text.
   - Below the heading, provide **3 destination images** (e.g., Paris, Tokyo, Sydney). Each image should:
     - Have a border of 5px solid dark green.
     - A border-radius of 10px.
     - Be linked to a relevant webpage/resource about the destination.
     - The images should be displayed side-by-side in a responsive manner with a 20px gap.
   - Each image should have a caption under it that says the name of the location, with the text centered and font size of 18px.
5. **Travel Tips Section:**

- An overridden displayDetails() method to print the inpatient bill details.

3. **Derived Class: OutpatientBill**

- This class will inherit from Bill and represent an outpatient bill, which includes additional information like doctor's fees and lab charges.
- It will have the following properties:
  - patientID: A unique identifier for the outpatient (e.g., "P007").
  - doctorFee: The fee charged by the doctor for consultation.
  - labCharges: Any additional charges for laboratory tests.

Methods:

- Overloaded setDetails() methods for setting the outpatient bill details (amount, patient ID, doctor's fee, and lab charges).
- An overridden displayDetails() method to print the outpatient bill details.

4. **Main Function:**

- The main program should create objects of both InpatientBill and OutpatientBill using pointers to the base class Bill.
- The user should be prompted to input the required details (such as the amount, patient ID, room type, days admitted, etc.).
- Once the details are input, the processBill() method should be called to display the details of both inpatient and outpatient bills.

Output

```
Enter Inpatient Bill Details:
Amount: 2000
Patient ID: P1234
Room Type (e.g., General, Private): Private
Days Admitted: 3

Enter Outpatient Bill Details:
Amount: 800
Patient ID: P5678
Doctor Fee: 300
Lab Charges: 100

Inpatient Bill set for Patient ID P1234 with room type Private.
Number of days admitted: 3
Processing Bill...
Inpatient Bill Details:
  Amount: 2000
  Patient ID: P1234
  Room Type: Private
  Days Admitted: 3

Outpatient Bill set for Patient ID P5678 with doctor fee 300
Lab charges: 100
Processing Bill...
Outpatient Bill Details:
  Amount: 800
  Patient ID: P5678
  Doctor Fee: 300
  Lab Charges: 100
```

- Use **inheritance** to define common and specific properties for full-time and part-time employees.
- Use **polymorphism** to calculate and display salaries for different employee types.
- Store the employees in an array and allow the user to interactively add employees and view the details.

Output

```
Employee Management System
1. Add Full Time Employee
2. Add Part Time Employee
3. Display All Employees
4. Exit
Enter your choice: 1
Enter Full Time Employee Details
Name: John
ID: [illegible]
Department: IT
Bonus: [illegible]

Employee Management System
1. Add Full Time Employee
2. Add Part Time Employee
3. Display All Employees
4. Exit
Enter your choice: 2
Enter Part Time Employee Details
Name: Jane
ID: [illegible]
Department: HR
Hourly Rate: [illegible]
Hours Worked: [illegible]

Employee Management System
1. Add Full Time Employee
2. Add Part Time Employee
3. Display All Employees
4. Exit
Enter your choice: 3

Employee Details
Name: John, ID: [illegible], Department: IT, Salary: [illegible]
Bonus: [illegible], Total Salary: [illegible]
---------------------
Name: Jane, ID: [illegible], Department: HR, Salary: [illegible]
Hourly Rate: [illegible], Hours Worked: [illegible]
--------------------- [illegible]
```

<<<<<<<<<< Best of Luck :) >>>>>>>>>>

- The H4 heading should read **"Essential Travel Tips"** with purple text, a font size of 30px, and bold text.
- Below the heading, provide a list of 5 travel tips using an unordered list:
  - Each list item should be styled with a small icon (use emojis or placeholder icons).
  - The font size for the list items should be 18px with 10px margin at the bottom.

6. **Connect Section:**
   - The H5 heading should read **"Connect with Fellow Travelers"** with dark red text, a font size of 28px, center-aligned text, and a yellow background.
   - Below the heading, provide a **link to a travel community resource** (e.g., Lonely Planet) with a relevant image (e.g., a group of travelers).
   - The image should have a solid border and padding of 10px.
7. **Login Section:**
   - The H6 heading should read **"Ready to Start Your Journey?"** with dark blue text, a font size of 26px, and centered alignment.
   - Below the heading, provide a **Login button** styled as a link with a background color of orange (#ff8000), white text, and padding of 10px 20px. The button should be clearly visible and placed at the center of the section.
   - The button should be an anchor link that directs the user to a login page (e.g., https://example.com/login).
8. **Footer:**
   - The footer should contain a copyright notice **"© 2024 Discover New Destinations. All rights reserved."** styled with white text and a background color of dark blue.
   - The font size should be 16px, and the text should be centered.

Q3: **[wgt: 20]**

You are tasked with creating an **Employee Management System** using Object-Oriented Programming (OOP) principles. The system will manage two types of employees: **Full-time** and **Part-time** employees. The program should use **classes** and support **inheritance** and **polymorphism.**

Requirements:

1. **Classes and Data Members:**
   - **Base Class: Employee**
     - Attributes: name, id, department, and salary.
     - A **virtual function** calculateSalary() to be overridden in derived classes.
     - A **function** displayDetails() to print employee details.
   - **Derived Class: FullTimeEmployee** (inherits from Employee)
     - Additional attribute: bonus (for the bonus of full-time employees).
     - Overriding calculateSalary() to include a fixed salary plus bonus.
   - **Derived Class: PartTimeEmployee** (inherits from Employee)
     - Additional attributes: hourlyRate and hoursWorked.
     - Overriding calculateSalary() to calculate salary based on hours worked.
2. **Functions:**

- **addFullTimeEmployee()**: Adds a full-time employee by collecting input for name, id, department, and bonus.
- **addPartTimeEmployee()**: Adds a part-time employee by collecting input for name, id, department, hourly rate, and hours worked.
- **displayAllEmployees()**: Displays the details of all employees, including their salary calculated by the respective calculateSalary() method.

3. **Program Flow:**

National University of Computer and Emerging Sciences

Karachi – Campus

Introduction to Information and Communication Technologies (CL1000)

Lab Final-Exam

Total Time (Hrs): 02

Total Marks: 50

Total Questions: 04

Paper B

Date: December 05, 2024

Time: 08:30 PM – 10:30 AM

Course Instructor(s)

Mr. Ghulam Qadir Bhurgari, Miss. Khadija tul Kubra

[redacted]

Student Name Section Roll Number

Q1: **[wgt: 10]**

You are tasked with creating an **Employee Performance and Salary Management System** to manage employees, track their performance, and calculate their salaries. The system will allow you to add new employees, update their performance scores, and calculate their salary based on the performance.

Requirements:

1. **Data Structure**:
   - Use an array to store the details of employees.
   - Each employee has the following attributes:
     - name (string): Employee's name.
     - performanceScore (number): Score representing the performance (out of 100).
     - baseSalary (number): The base salary of the employee.
     - bonus (number): Additional salary based on performance score.
     - totalSalary (number): Total salary including base salary and bonus.
     - status (string): Employee's status, either "Active" or "Inactive".
2. **Functions**:
   - **addEmployee(name, baseSalary, performanceScore)**: Adds a new employee with their name, base salary, and performance score.
   - **updatePerformance(index, newPerformanceScore)**: Updates the performance score of a specific employee by their index in the array.
   - **calculateBonus(index)**: Calculates and updates the bonus based on the performance score. If the score is 90 or above, the bonus is 20% of the base salary; if the score is between 70 and 89, the bonus is 10% of the base salary; otherwise, no bonus.
   - **calculateTotalSalary(index)**: Calculates the total salary of an employee, which includes the base salary and the bonus.
   - **displayEmployeeDetails()**: Displays all employees' details: name, performance score, base salary, bonus, total salary, and status.
3. **Program Flow**:
   - Add employees using the addEmployee function.
   - Update employees' performance scores using the updatePerformance function.
   - Calculate the bonus and total salary using calculateBonus and calculateTotalSalary functions.

- Display all employees' details using displayEmployeeDetails.

Output

```
Employee added: Alice, Base Salary: 50000, Performance Score: 85, Status: Active
Employee added: Bob, Base Salary: 45000, Performance Score: 70, Status: Active
Employee updated: Alice's new performance score is 92, Bonus: 10000, Total Salary: 60000
Employee updated: Bob's new performance score is 65, No Bonus, Total Salary: 45000

All Employees
1. Alice, Performance Score: 92, Base Salary: 50000, Bonus: 10000, Total Salary: 60000, Status: Active
2. Bob, Performance Score: 65, Base Salary: 45000, Bonus: 0, Total Salary: 45000, Status: Active
```

Q2: **[wgt: 15]**

You are required to create a **restaurant menu webpage** using HTML and inline CSS. First, create a file called restaurant_menu.html with a complete HTML structure and title the page as **"Welcome to Our Restaurant Menu"**. The page should contain several sections, each with specific styles applied directly through inline CSS. The webpage should include the following:

1. **Hero Section**:
   - The H1 heading should read **"Welcome to Our Restaurant Menu"** with the following style:
     - Dark red text
     - Font size of 48px
     - Center-aligned text
     - 50px top margin
     - A background image of a food-related theme should cover the entire section.
2. **Navigation Bar**:
   - The navigation bar should contain several anchor links to different sections on the page (e.g., Appetizers, Main Course, Desserts, Drinks, Contact). Ensure the following:
     - Each link should have a 15px margin.
     - Use a dark brown background for the navbar.
     - The **Contact button** should be highlighted in bold with a background color of golden yellow (**#ffcc00**) and padding of 8px 16px. It should be positioned on the right side of the navbar.
     - Use inline CSS to style the links with appropriate font sizes (18px) and colors (white).
3. **Appetizers Section**:
   - The H2 heading should read **"Appetizers"** with dark green text, a font size of 36px, and center-aligned text.
   - The section should have a light beige background with 40px padding around the content.
   - Below the heading, list **at least 5 appetizer items**, each with a description, price, and a small image.
     - Each item should be styled with a font size of 20px and a 15px margin at the bottom.
     - Use inline CSS to style the images with a width of 150px and a 2px solid border.
4. **Main Course Section**:
   - The H3 heading should read **"Main Course"** with dark blue text, a font size of 32px, and italicized text.
   - Below the heading, list **at least 5 main course items**, each with a description, price, and a small image.
     - The items should have a font size of 20px and a 10px bottom margin.
     - Each image should have a border of 3px solid dark blue and a border-radius of 5px.
5. **Desserts Section**:
   - The H4 heading should read **"Desserts"** with purple text, a font size of 30px, and bold text.

- Below the heading, list **at least 5 dessert items** [illegible]
  - Items should be styled with a font size of [illegible]
  - Use inline CSS to ensure the images have a [illegible]

6. **Drinks Section**:
   - The H5 heading should read "**Drinks**" with dark orange [illegible] aligned text.
   - Below the heading, list **at least 4 drink items** [illegible]
     - The items should be styled with a font size of [illegible]
     - Each drink image should have a 2px solid orange border [illegible]
7. **Contact Section**:
   - The H6 heading should read "**Contact Us**" with dark brown text, [illegible] alignment.
   - Below the heading, provide a **contact form** with the following fields:
     - Name (text input)
     - Email (email input)
     - Message (textarea)
     - Submit button with a background color of green [illegible]
   - Style the form fields and submit button using inline CSS [illegible]
8. **Footer**
   - The footer should contain the restaurant's contact information (phone number, address, and email).
   - The text should be white with a dark gray background color.
   - Include links to the restaurant's social media pages (e.g., Facebook, Instagram, [illegible]) with icons or emojis, each with a 10px margin between them.

Q3. [illegible]

You are tasked with designing a **Hospital Billing System** that can handle both inpatient and outpatient billing using Object-Oriented Programming (OOP) principles. The system should manage patient billing details with various charges, and display accurate billing information for both inpatient and outpatient cases.

The system should include the following:

1. **Base Class: Bill**
   - This class represents the general billing details for any type of patient (inpatient or outpatient).
   - It will have the following properties:
     - amount: Represents the total bill amount.
     - Methods:
       - setDetails(): Overloaded methods for setting the amount, which may include a description.
       - displayDetails(): A method to display the details of the bill.
       - processBill(): A method that processes the bill and displays the bill details.
2. **Derived Class: InpatientBill**
   - This class will inherit from Bill and represent an inpatient bill, which includes additional information like room type and days admitted.
   - It will have the following properties:
     - patientID: A unique identifier for the patient (e.g., "P123").
     - roomType: The type of room the patient is assigned to (e.g., "Private", "General").
     - daysAdmitted: The number of days the patient was admitted to the hospital.
   - Methods:
     - Overloaded setDetails() methods for setting the inpatient bill details (amount, patient ID, room type, days admitted).